Zmienna liczba argumentów i dowolne argumenty nazwane

Zmienna liczba argumentów pozycyjnych

Niektóre funkcje mogą przyjąć dowolną liczbę argumentów pozycyjnych. Przykładem takiej funkcji jest `print`:

In [2]:
```
print()
```

In [3]:
```
print(1, 2, 3)
```

```
1 2 3
```

In [4]:
```
print("Stoi", "na", "stacji", "lokomotywa")
```

```
Stoi na stacji lokomotywa
```

Parametr poprzedzony gwiazdką w definicji funkcji „zbiera” wszystkie końcowe argumenty pozycyjne jako krotkę. Dla poniższej definicji wszystkie argumenty poza pierwszym będą dostępne jako krotka `xs`.

In [5]:
```
def srednia_arytmetyczna(x, *xs):
    print(f"x = {x}, xs = {xs}")
    return (x + sum(xs)) / (1 + len(xs))
```

In [6]:
```
srednia_arytmetyczna(1.0, 2.0, 3.0, 4.0)
```

```
x = 1.0, xs = (2.0, 3.0, 4.0)
```

Out[6]:
```
2.5
```

In [7]:
```
srednia_arytmetyczna(1.0)
```

```
x = 1.0, xs = ()
```

Out[7]:
```
1.0
```

Parametr `x` wymaga przynajmniej jednego argumentu pozycyjnego. Wywołanie bez argumentów nie poda jego wartości i spowoduje wyjątek `TypeError`:

In [8]:
```
srednia_arytmetyczna()
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[8], line 1
----> 1 srednia_arytmetyczna()

TypeError: srednia_arytmetyczna() missing 1 required positional argument:
'x'
```

Wszystkie argumenty, które na liście parametrów są za parametrem z gwiazdką muszą być przekazane jako nazwane (inaczej byłyby też „wyłapane” przez parametr z gwiazdką), gwiazdka działa tu też tak jak sama gwiazdka na liście parametrów.

Wypróbujmy zmodyfikowaną definicję funkcji `srednia_arytmetyczna` z opcjonalnym wyświetlaniem wartości parametrów:

In [9]:
```
def srednia_arytmetyczna(x, *xs, wyswietl=False):
    if wyswietl:
        print(f"x = {x}, xs = {xs}")
    return (x + sum(xs)) / (1 + len(xs))
```

Wartość parametru `wyswietl` musi być przekazana jako argument nazwany. Poniższy przykład potraktuje argument `True` jako wartość czwartą wartość liczbową równą 1.

In [10]:
```
srednia_arytmetyczna(10, 30, 40, True)
```

Out[10]:
```
20.25
```

Jedyny sposób, to:

In [11]:
```
srednia_arytmetyczna(10, 30, 40, wyswietl=True)
```

```
x = 10, xs = (30, 40)
```

Out[11]:
```
26.666666666666668
```

Parametr `*xs` już wyznacza granicę, po której wartości parametrów trzeba przekazywać jako argumenty nazwane. Nie dodajemy za nim osobnego separatora `*`: poniższa próba zdefiniowania funkcji z obiema konstrukcjami jest niepoprawna składniowo i zgłosi wyjątek `SyntaxError`:

In [12]:
```
def srednia_arytmetyczna(x, *xs, *, wyswietl=False):
    if wyswietl:
        print(f"x = {x}, xs = {xs}")
    return (x + sum(xs)) / (1 + len(xs))
```

```
  Cell In[12], line 1
    def srednia_arytmetyczna(x, *xs, *, wyswietl=False):
                                     ^
SyntaxError: * may appear only once
```

Rozwijanie sekwencji do listy argumentów

Elementy sekwencji można w łatwy sposób przekazać jako kolejne argumenty wywołania funkcji. Gwiazdka użyta przed argumentem będącym sekwencją „rozbije” tę sekwencję na kolejne parametry.

Mamy funkcję, która zwraca sumę swoich trzech parametrów:

In [13]:
```
def suma_3(a, b, c):
    return a + b + c
```

Mamy sekwencję (np. listę) z trzema elementami do zsumowania:

In [14]:
```
skladniki = [1, 2, 3]
```

Używając powyższej funkcji możemy to zrobić np. tak:

In [15]:
```
suma_3(skladniki[0], skladniki[1], skladniki[2])
```

Out[15]: 6

Funkcja `suma_3` wymaga trzech argumentów. Poniżej przekazujemy tylko jeden — listę z trzema elementami. Python nie przypisze automatycznie jej elementów do trzech parametrów, więc brak wartości dla dwóch z nich spowoduje wyjątek `TypeError`:

In [16]:
```
suma_3(skladniki)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[16], line 1
----> 1 suma_3(skladniki)

TypeError: suma_3() missing 2 required positional arguments: 'b' and 'c'
```

Możemy jednak zrobić to tak jak poniżej. Wszystkie elementy sekwencji zostaną użyte jako kolejne argumenty pozycyjne funkcji:

In [17]:
```
suma_3(*skladniki)
```

Out[17]: 6

In [18]:
```
skladniki.append(4)
```

Po dopisaniu elementu lista `skladniki` ma już cztery elementy. Gwiazdka przekaże je jako cztery osobne argumenty pozycyjne, ale funkcja `suma_3` przyjmuje tylko trzy. To wywołanie zakończy się wyjątkiem `TypeError`:

In [19]:
```
suma_3(*skladniki)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[19], line 1
----> 1 suma_3(*skladniki)

TypeError: suma_3() takes 3 positional arguments but 4 were given
```

Szczególnie przydatne jest to, gdy chcemy użyć funkcji, która przyjmuje dowolną liczbę argumentów, a elementy są przechowywane w sekwencji, ale nie jest znana jej długość, lub sekwencja jest długa. Jeżeli długość nie jest znana, to nie możemy przekazać wszystkich argumentów osobno. Jeżeli sekwencja jest długa, to jest to niepraktyczne.

Przykład:

Powtarzamy potrzebną definicję średniej, aby ten przykład można było wykonać bez szukania jej w poprzedniej części:

In [20]:
```
def srednia_arytmetyczna(x, *xs, wyswietl=False):
    if wyswietl:
        print(f"x = {x}, xs = {xs}")
    return (x + sum(xs)) / (1 + len(xs))
```

In [21]:
```
xs = [i**2 for i in range(1000)]
```

Teoretycznie da się użyć wywołania `srednia_arytmetyczna(xs[0], ... xs[999])`, ale byłoby to wywołanie z tysiącem argumentów. Jest to kompletnie niepraktyczne i oczywiście wygodniej użyć poniższego wywołania:

In [22]:
```
srednia_arytmetyczna(*xs, wyswietl=True)
```

x = 0, xs = (1, 4, 9, 16, 25, 36, 49, 64, 81, 100, 121, 144, 169, 196, 225,
256, 289, 324, 361, 400, 441, 484, 529, 576, 625, 676, 729, 784, 841, 900, 9
61, 1024, 1089, 1156, 1225, 1296, 1369, 1444, 1521, 1600, 1681, 1764, 1849,
1936, 2025, 2116, 2209, 2304, 2401, 2500, 2601, 2704, 2809, 2916, 3025, 313
6, 3249, 3364, 3481, 3600, 3721, 3844, 3969, 4096, 4225, 4356, 4489, 4624, 4
761, 4900, 5041, 5184, 5329, 5476, 5625, 5776, 5929, 6084, 6241, 6400, 6561,
6724, 6889, 7056, 7225, 7396, 7569, 7744, 7921, 8100, 8281, 8464, 8649, 883
6, 9025, 9216, 9409, 9604, 9801, 10000, 10201, 10404, 10609, 10816, 11025, 1
1236, 11449, 11664, 11881, 12100, 12321, 12544, 12769, 12996, 13225, 13456,
13689, 13924, 14161, 14400, 14641, 14884, 15129, 15376, 15625, 15876, 16129,
16384, 16641, 16900, 17161, 17424, 17689, 17956, 18225, 18496, 18769, 19044,
19321, 19600, 19881, 20164, 20449, 20736, 21025, 21316, 21609, 21904, 22201,
22500, 22801, 23104, 23409, 23716, 24025, 24336, 24649, 24964, 25281, 25600,
25921, 26244, 26569, 26896, 27225, 27556, 27889, 28224, 28561, 28900, 29241,
29584, 29929, 30276, 30625, 30976, 31329, 31684, 32041, 32400, 32761, 33124,
33489, 33856, 34225, 34596, 34969, 35344, 35721, 36100, 36481, 36864, 37249,
37636, 38025, 38416, 38809, 39204, 39601, 40000, 40401, 40804, 41209, 41616,
42025, 42436, 42849, 43264, 43681, 44100, 44521, 44944, 45369, 45796, 46225,
46656, 47089, 47524, 47961, 48400, 48841, 49284, 49729, 50176, 50625, 51076,
51529, 51984, 52441, 52900, 53361, 53824, 54289, 54756, 55225, 55696, 56169,
56644, 57121, 57600, 58081, 58564, 59049, 59536, 60025, 60516, 61009, 61504,
62001, 62500, 63001, 63504, 64009, 64516, 65025, 65536, 66049, 66564, 67081,
67600, 68121, 68644, 69169, 69696, 70225, 70756, 71289, 71824, 72361, 72900,
73441, 73984, 74529, 75076, 75625, 76176, 76729, 77284, 77841, 78400, 78961,
79524, 80089, 80656, 81225, 81796, 82369, 82944, 83521, 84100, 84681, 85264,
85849, 86436, 87025, 87616, 88209, 88804, 89401, 90000, 90601, 91204, 91809,
92416, 93025, 93636, 94249, 94864, 95481, 96100, 96721, 97344, 97969, 98596,
99225, 99856, 100489, 101124, 101761, 102400, 103041, 103684, 104329, 10497
6, 105625, 106276, 106929, 107584, 108241, 108900, 109561, 110224, 110889, 1
11556, 112225, 112896, 113569, 114244, 114921, 115600, 116281, 116964, 11764
9, 118336, 119025, 119716, 120409, 121104, 121801, 122500, 123201, 123904, 1
24609, 125316, 126025, 126736, 127449, 128164, 128881, 129600, 130321, 13104
4, 131769, 132496, 133225, 133956, 134689, 135424, 136161, 136900, 137641, 1
38384, 139129, 139876, 140625, 141376, 142129, 142884, 143641, 144400, 14516
1, 145924, 146689, 147456, 148225, 148996, 149769, 150544, 151321, 152100, 1
52881, 153664, 154449, 155236, 156025, 156816, 157609, 158404, 159201, 16000
0, 160801, 161604, 162409, 163216, 164025, 164836, 165649, 166464, 167281, 1
68100, 168921, 169744, 170569, 171396, 172225, 173056, 173889, 174724, 17556
1, 176400, 177241, 178084, 178929, 179776, 180625, 181476, 182329, 183184, 1
84041, 184900, 185761, 186624, 187489, 188356, 189225, 190096, 190969, 19184
4, 192721, 193600, 194481, 195364, 196249, 197136, 198025, 198916, 199809, 2
00704, 201601, 202500, 203401, 204304, 205209, 206116, 207025, 207936, 20884
9, 209764, 210681, 211600, 212521, 213444, 214369, 215296, 216225, 217156, 2
18089, 219024, 219961, 220900, 221841, 222784, 223729, 224676, 225625, 22657
6, 227529, 228484, 229441, 230400, 231361, 232324, 233289, 234256, 235225, 2
36196, 237169, 238144, 239121, 240100, 241081, 242064, 243049, 244036, 24502
5, 246016, 247009, 248004, 249001, 250000, 251001, 252004, 253009, 254016, 2
55025, 256036, 257049, 258064, 259081, 260100, 261121, 262144, 263169, 26419
6, 265225, 266256, 267289, 268324, 269361, 270400, 271441, 272484, 273529, 2
74576, 275625, 276676, 277729, 278784, 279841, 280900, 281961, 283024, 28408
9, 285156, 286225, 287296, 288369, 289444, 290521, 291600, 292681, 293764, 2
94849, 295936, 297025, 298116, 299209, 300304, 301401, 302500, 303601, 30470
4, 305809, 306916, 308025, 309136, 310249, 311364, 312481, 313600, 314721, 3
15844, 316969, 318096, 319225, 320356, 321489, 322624, 323761, 324900, 32604
1, 327184, 328329, 329476, 330625, 331776, 332929, 334084, 335241, 336400, 3
37561, 338724, 339889, 341056, 342225, 343396, 344569, 345744, 346921, 34810

```
0, 349281, 350464, 351649, 352836, 354025, 355216, 356409, 357604, 358801, 3
60000, 361201, 362404, 363609, 364816, 366025, 367236, 368449, 369664, 37088
1, 372100, 373321, 374544, 375769, 376996, 378225, 379456, 380689, 381924, 3
83161, 384400, 385641, 386884, 388129, 389376, 390625, 391876, 393129, 39438
4, 395641, 396900, 398161, 399424, 400689, 401956, 403225, 404496, 405769, 4
07044, 408321, 409600, 410881, 412164, 413449, 414736, 416025, 417316, 41860
9, 419904, 421201, 422500, 423801, 425104, 426409, 427716, 429025, 430336, 4
31649, 432964, 434281, 435600, 436921, 438244, 439569, 440896, 442225, 44355
6, 444889, 446224, 447561, 448900, 450241, 451584, 452929, 454276, 455625, 4
56976, 458329, 459684, 461041, 462400, 463761, 465124, 466489, 467856, 46922
5, 470596, 471969, 473344, 474721, 476100, 477481, 478864, 480249, 481636, 4
83025, 484416, 485809, 487204, 488601, 490000, 491401, 492804, 494209, 49561
6, 497025, 498436, 499849, 501264, 502681, 504100, 505521, 506944, 508369, 5
09796, 511225, 512656, 514089, 515524, 516961, 518400, 519841, 521284, 52272
9, 524176, 525625, 527076, 528529, 529984, 531441, 532900, 534361, 535824, 5
37289, 538756, 540225, 541696, 543169, 544644, 546121, 547600, 549081, 55056
4, 552049, 553536, 555025, 556516, 558009, 559504, 561001, 562500, 564001, 5
65504, 567009, 568516, 570025, 571536, 573049, 574564, 576081, 577600, 57912
1, 580644, 582169, 583696, 585225, 586756, 588289, 589824, 591361, 592900, 5
94441, 595984, 597529, 599076, 600625, 602176, 603729, 605284, 606841, 60840
0, 609961, 611524, 613089, 614656, 616225, 617796, 619369, 620944, 622521, 6
24100, 625681, 627264, 628849, 630436, 632025, 633616, 635209, 636804, 63840
1, 640000, 641601, 643204, 644809, 646416, 648025, 649636, 651249, 652864, 6
54481, 656100, 657721, 659344, 660969, 662596, 664225, 665856, 667489, 66912
4, 670761, 672400, 674041, 675684, 677329, 678976, 680625, 682276, 683929, 6
85584, 687241, 688900, 690561, 692224, 693889, 695556, 697225, 698896, 70056
9, 702244, 703921, 705600, 707281, 708964, 710649, 712336, 714025, 715716, 7
17409, 719104, 720801, 722500, 724201, 725904, 727609, 729316, 731025, 73273
6, 734449, 736164, 737881, 739600, 741321, 743044, 744769, 746496, 748225, 7
49956, 751689, 753424, 755161, 756900, 758641, 760384, 762129, 763876, 76562
5, 767376, 769129, 770884, 772641, 774400, 776161, 777924, 779689, 781456, 7
83225, 784996, 786769, 788544, 790321, 792100, 793881, 795664, 797449, 79923
6, 801025, 802816, 804609, 806404, 808201, 810000, 811801, 813604, 815409, 8
17216, 819025, 820836, 822649, 824464, 826281, 828100, 829921, 831744, 83356
9, 835396, 837225, 839056, 840889, 842724, 844561, 846400, 848241, 850084, 8
51929, 853776, 855625, 857476, 859329, 861184, 863041, 864900, 866761, 86862
4, 870489, 872356, 874225, 876096, 877969, 879844, 881721, 883600, 885481, 8
87364, 889249, 891136, 893025, 894916, 896809, 898704, 900601, 902500, 90440
1, 906304, 908209, 910116, 912025, 913936, 915849, 917764, 919681, 921600, 9
23521, 925444, 927369, 929296, 931225, 933156, 935089, 937024, 938961, 94090
0, 942841, 944784, 946729, 948676, 950625, 952576, 954529, 956484, 958441, 9
60400, 962361, 964324, 966289, 968256, 970225, 972196, 974169, 976144, 97812
1, 980100, 982081, 984064, 986049, 988036, 990025, 992016, 994009, 996004, 9
98001)
```

Out[22]: 332833.5

Kolejne przykłady użycia gwiazdki przed argumentem wywołania:

In [23]:
```
srednia_arytmetyczna(0, *skladniki, 4, wyswietl=True)
```

```
x = 0, xs = (1, 2, 3, 4, 4)
```

Out[23]: 2.3333333333333335

In [24]:
```
srednia_arytmetyczna(*range(5, 8), wyswietl=True)
```

```
x = 5, xs = (6, 7)
```

Out[24]:
```
6.0
```

In [25]:
```
srednia_arytmetyczna(*range(10), 10, *skladniki, 11, wyswietl=True)
```

```
x = 0, xs = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 1, 2, 3, 4, 11)
```

Out[25]:
```
4.75
```

(*) Przykład z funkcją `zip` (i brakiem funkcji `unzip`):

Funkcja `zip` pozwala połączyć osobne sekwencje w pojedynczą sekwencję krotek. Pierwsze elementy wszystkich sekwencji utworzą pierwszą krotkę, drugie — drugą, itd.

In [26]:
```
numery = [1, 2, 3, 4]
nazwy = ["Świdnik", "Lublin", "Warszawa", "Tokio"]
populacja = [40_000, 400_000, 1_600_000, 10_000_000]
```

In [27]:
```
miasta = list(zip(numery, nazwy, populacja))
```

In [28]:
```
miasta
```

Out[28]:
```
[(1, 'Świdnik', 40000),
 (2, 'Lublin', 400000),
 (3, 'Warszawa', 1600000),
 (4, 'Tokio', 10000000)]
```

W bibliotece standardowej nie ma funkcji odwrotnej, czyli funkcji, która rozbija listę takich krotek np. na krotkę osobnych sekwencji.

Ale zastanówmy się jaki wynik da takie wywołanie:

In [29]:
```
list(
    zip(
        (1, "Świdnik", 40000),
        (2, "Lublin", 400000),
        (3, "Warszawa", 1600000),
        (4, "Tokio", 10000000),
    )
)
```

Out[29]:
```
[(1, 2, 3, 4),
 ('Świdnik', 'Lublin', 'Warszawa', 'Tokio'),
 (40000, 400000, 1600000, 10000000)]
```

Przy zamianie sekwencji numerów, nazw i populacji na sekwencję miast (krotek) wywołanie `zip` ma zawsze trzy argumenty: numery, nazwy i populacje. Przy zamianie w odwrotną stronę — miast na trzy osobne sekwencje — liczba argumentów funkcji `zip` to liczba miast. Jeżeli więc mamy miasta w liście, to musimy właśnie użyć argumentu z gwiazdką:

In [30]:
```
list(zip(*miasta))
```

Out[30]:
```
[(1, 2, 3, 4),
 ('Świdnik', 'Lublin', 'Warszawa', 'Tokio'),
 (40000, 400000, 1600000, 10000000)]
```

Nie jest to do końca operacja odwrotna — w wyniku mamy listę krotek, a początkowo dane były w listach, ale w razie potrzeby możemy sobie te krotki łatwo skonwertować na listy.

Funkcje przyjmujące dowolne argumenty nazwane

W podobny sposób jak możemy napisać funkcję, która przyjmuje dowolną liczbę parametrów pozycyjnych używając gwiazdki przed jednym z parametrów, tak samo możemy napisać funkcję, która przyjmie dowolne argumenty nazwane używając dwóch gwiazdek przed parametrem. Wartością tego parametru będzie wtedy słownik. Kluczami w tym słowniku będą nazwy argumentów, a wartościami — wartości tych argumentów.

Przykład: Funkcja `filtruj` filtruje listę słowników według podanych kryteriów. Zwraca tylko te elementy, w których pola zadane argumentami nazwanymi mają zadaną wartość.

In [31]:
```
def filtruj(dane, **filtry):
    print(f"filtry: {filtry}")
    pasujace = []
    for element in dane:
        pasuje = True
        for nazwa, wartosc in filtry.items():
            if element[nazwa] != wartosc:
                pasuje = False
                break
        if pasuje:
            pasujace.append(element)
    return pasujace
```

Przykłady użycia:

In [32]:
```
produkty = [
    {"nazwa": "krzesło", "typ": "mebel", "cena": 300},
    {"nazwa": "biurko", "typ": "mebel", "cena": 2000},
    {"nazwa": "rower", "typ": "sprzęt sportowy", "cena": 2000},
    {"nazwa": "piłka", "typ": "sprzęt sportowy", "cena": 200},
]

uczniowie = [
    {"imie": "Jan", "nazwisko": "Kowalski", "ocena": 5},
    {"imie": "Anna", "nazwisko": "Nowak", "ocena": 3},
```

```
    {"imie": "Piotr", "nazwisko": "Kowalski", "ocena": 2},
    {"imie": "Joanna", "nazwisko": "Wójtowicz", "ocena": 5},
    {"imie": "Piotr", "nazwisko": "Wójtowicz", "ocena": 3},
    {"imie": "Piotr", "nazwisko": "Nowak", "ocena": 3},
    {"imie": "Beata", "nazwisko": "Wójcik", "ocena": 4},
]
```

In [33]:
```
filtruj(produkty, typ="mebel")
```

```
filtry: {'typ': 'mebel'}
```

Out[33]:
```
[{'nazwa': 'krzesło', 'typ': 'mebel', 'cena': 300},
 {'nazwa': 'biurko', 'typ': 'mebel', 'cena': 2000}]
```

In [34]:
```
filtruj(produkty, typ="mebel", cena=2000)
```

```
filtry: {'typ': 'mebel', 'cena': 2000}
```

Out[34]:
```
[{'nazwa': 'biurko', 'typ': 'mebel', 'cena': 2000}]
```

In [35]:
```
filtruj(uczniowie, imie="Piotr")
```

```
filtry: {'imie': 'Piotr'}
```

Out[35]:
```
[{'imie': 'Piotr', 'nazwisko': 'Kowalski', 'ocena': 2},
 {'imie': 'Piotr', 'nazwisko': 'Wójtowicz', 'ocena': 3},
 {'imie': 'Piotr', 'nazwisko': 'Nowak', 'ocena': 3}]
```

In [36]:
```
filtruj(uczniowie, imie="Piotr", ocena=3)
```

```
filtry: {'imie': 'Piotr', 'ocena': 3}
```

Out[36]:
```
[{'imie': 'Piotr', 'nazwisko': 'Wójtowicz', 'ocena': 3},
 {'imie': 'Piotr', 'nazwisko': 'Nowak', 'ocena': 3}]
```

In [37]:
```
filtruj(uczniowie, ocena=6)
```

```
filtry: {'ocena': 6}
```

Out[37]:
```
[]
```

In [38]:
```
filtruj(produkty)
```

```
filtry: {}
```

Out[38]:
```
[{'nazwa': 'krzesło', 'typ': 'mebel', 'cena': 300},
 {'nazwa': 'biurko', 'typ': 'mebel', 'cena': 2000},
 {'nazwa': 'rower', 'typ': 'sprzęt sportowy', 'cena': 2000},
 {'nazwa': 'piłka', 'typ': 'sprzęt sportowy', 'cena': 200}]
```

Nie możemy tutaj użyć w definicji konkretnych parametrów dla każdego filtra, bo ani nazwy filtrów ani ich liczba nie są w żaden sposób ograniczone.

Podobny mechanizm przyjmowania dowolnych argumentów nazwanych wykorzystuje metoda `dict.update`. Możemy przekazać jej słownik z nowymi wpisami albo podać wpisy jako argumenty nazwane. Nazwy argumentów stają się kluczami, a podane wartości — wartościami wpisów:

In [39]:
```
d = {"a": 1, "b": 2}
```

In [40]:
```
d.update({"b": 3, "c": 4})
```

In [41]:
```
d.update(c=10, d=11)
```

In [42]:
```
d
```

Out[42]:
```
{'a': 1, 'b': 3, 'c': 10, 'd': 11}
```

Rozwijanie słowników do argumentów nazwanych

W taki sam sposób jak wartości parametrów pozycyjnych można wziąć z sekwencji poprzedzonej gwiazdką, tak wartości parametrów nazwanych można wziąć ze słownika (lub innego tzw. **mapującego** typu danych, ang. *mapping*) poprzedzonego dwiema gwiazdkami. Na przykład parametry `sep` i `end` funkcji `print` możemy przekazać w następujący sposób:

In [43]:
```
parametry = {"sep": " + ", "end": ";"}
print(1, 2, 3, **parametry)
```

```
1 + 2 + 3;
```

Jest to równoważne wywołaniu:

In [44]:
```
print(1, 2, 3, sep=" + ", end=";")
```

```
1 + 2 + 3;
```

W powyższym przykładzie nie widać korzyści z użycia tego mechanizmu. Wersja z jawnym użyciem argumentów nazwanych jest prostsza. Rozważmy teraz bardziej praktyczny przykład użycia.

Załóżmy, że nasz program przechowuje pewne dane w słowniku. Chcemy napisać funkcję zapisującą te dane do pliku w formacie JSON. Chcemy umożliwić dokładną konfigurację zapisu danych za pomocą parametrów nazwanych:

- czy wynikowy plik ma używać do wcięć spacji zamiast tabulatorów (parametr logiczny `spacje`, domyślnie `True`),
- jeżeli używamy spacji, to ilu spacji użyć zamiast jednego tabulatora (parametr `rozmiar_tab`, domyślnie `4`),
- jaka jest nazwa pliku (parametr `nazwa_pliku`, domyślnie `wyniki.json`),
- czy klucze słownika mają być w wynikowym pliku posortowane (parametr `sortuj_klucze`, domyślnie `True`),
- czy plik wynikowy ma używać zestawu znaków ograniczonego do ASCII, czyli np. napis `współdzielony` zostanie zapisany jako `wsp\u00f3\u0142dzielony` (parametr logiczny `ascii`, domyślnie `False`).

Funkcja używa modułu `json` do konwersji skonstruowania danych w formacie JSON.

In [45]:
```
import json


def zapisz_wyniki(
    dane,
    spacje=True,
    rozmiar_tab=4,
    nazwa_pliku="wyniki.json",
    sortuj_klucze=True,
    ascii=False,
):
    if spacje:
        indentacja = rozmiar_tab
    else:
        indentacja = "\t"
    with open(nazwa_pliku, "w", encoding="utf-8") as p:
        json.dump(
            dane,
            p,
            ensure_ascii=ascii,
            indent=indentacja,
            sort_keys=sortuj_klucze,
        )
```

Przykład użycia funkcji:

In [46]:
```
dane = {
    "id": 123,
    "nazwisko": "Kowalski",
    "imię": "Jan",
    "oceny": [3, 3, 2, 5],
}
```

In [47]:
```
zapisz_wyniki(
    dane, rozmiar_tab=12, nazwa_pliku="jan_kowalski.json", ascii=True
)
```

Sprawdźmy co zostało zapisane do pliku:

In [48]:
```
with open("jan_kowalski.json") as p:
    print(p.read())
```

```
{
            "id": 123,
            "imi\u0119": "Jan",
            "nazwisko": "Kowalski",
            "oceny": [
                        3,
                        3,
                        2,
                        5
            ]
}
```

Załóżmy, że konfigurację w naszym programie przechowujemy w pliku w formacie TOML. Po wczytaniu danych z pliku mogą być one przechowywane jako napis:

In [49]:
```
konfiguracja_toml = (
    "spacje = true\n"
    "rozmiar_tab = 4\n"
    'nazwa_pliku = "dane_osobowe.json"\n'
    "sortuj_klucze = true\n"
    "ascii = false"
)
```

In [50]:
```
print(konfiguracja_toml)
```

```
spacje = true
rozmiar_tab = 4
nazwa_pliku = "dane_osobowe.json"
sortuj_klucze = true
ascii = false
```

Nazwy opcji konfiguracji są takie same jak nazwy parametrów funkcji `zapisz_wyniki`.

Moduł `tomllib` umożliwia łatwą zamianę takich danych na odpowiedni słownik:

In [51]:
```
import tomllib
```

In [52]:
```
konfiguracja = tomllib.loads(konfiguracja_toml)
```

In [53]:
```
konfiguracja
```

Out[53]:
```
{'spacje': True,
 'rozmiar_tab': 4,
 'nazwa_pliku': 'dane_osobowe.json',
 'sortuj_klucze': True,
 'ascii': False}
```

Możemy teraz wywołać funkcję `zapisz_wyniki` w taki sposób:

In [54]:
```
zapisz_wyniki(
    dane,
```

```
    spacje=konfiguracja["spacje"],
    rozmiar_tab=konfiguracja["rozmiar_tab"],
    nazwa_pliku=konfiguracja["nazwa_pliku"],
    sortuj_klucze=konfiguracja["sortuj_klucze"],
    ascii=konfiguracja["ascii"],
)
```

Ale przy użyciu opisanego mechanizmu będzie to wyglądać tak:

In [55]:
```
zapisz_wyniki(dane, **konfiguracja)
```

Funkcja przyjmująca dowolne argumenty

Do przykładów poniżej przydadzą się zdefiniowane wcześniej funkcje `srednia` i `srednia_arytmetyczna`. Powtarzamy tu ich definicje:

In [56]:
```
def srednia(dane, /, wagi=None, *, dokladnosc=None):
    if wagi is None:
        wagi = [1] * len(dane)
    wynik = sum(d * w for d, w in zip(dane, wagi)) / sum(wagi)
    if dokladnosc is not None:
        wynik = round(wynik, dokladnosc)
    return wynik
```

In [57]:
```
def srednia_arytmetyczna(x, *xs, wyswietl=False):
    if wyswietl:
        print(f"x = {x}, xs = {xs}")
    return (x + sum(xs)) / (1 + len(xs))
```

Poniżej podany jest przykład funkcji, która używa wszystkich czterech opisanych powyżej mechanizmów. Funkcja może przyjąć zarówno dowolną liczbę argumentów pozycyjnych jak i dowolne argumenty nazwane. Funkcja wywołuje inną funkcję i przekazuje jej otrzymaną krotkę jako osobne argumenty pozycyjne, oraz otrzymany słownik jako osobne argumenty nazwane.

Jest to przykład tzw. funkcji *opakowującej* inną funkcję. Funkcja dostaje jako pierwszy argument inną funkcję a jako kolejne argumenty dostaje argumenty do wywołania tej funkcji (zarówno pozycyjne jak i nazwane). Funkcja po prostu wywołuje tę funkcję używając zadanych parametrów, a oprócz tego wyświetla komunikat o wywołaniu tej funkcji, jej argumentach i wyniku.

Nazwy `args` i `kwargs` są powszechnie używanymi nazwami dla tego typu parametrów.

In [58]:
```
def wywolaj_funkcje(f, *args, **kwargs):
    print(f"wywołanie funkcji '{f.__name__}'")
    print(f"argumenty pozycyjne: {args}")
    print(f"argumenty nazwane: {kwargs}")
    wynik = f(*args, **kwargs)
```

```
    print(f"wynik = {wynik}")
    return wynik
```

In [59]:
```
from math import sin, sqrt, pi
```

In [60]:
```
wywolaj_funkcje(sin, pi / 2)
```

```
wywołanie funkcji 'sin'
argumenty pozycyjne: (1.5707963267948966,)
argumenty nazwane: {}
wynik = 1.0
```

Out[60]:
```
1.0
```

In [61]:
```
wywolaj_funkcje(srednia_arytmetyczna, 2, 3, 4, 5)
```

```
wywołanie funkcji 'srednia_arytmetyczna'
argumenty pozycyjne: (2, 3, 4, 5)
argumenty nazwane: {}
wynik = 3.5
```

Out[61]:
```
3.5
```

In [62]:
```
wywolaj_funkcje(srednia, [2, 3, 4], wagi=[1, 2, 4], dokladnosc=3)
```

```
wywołanie funkcji 'srednia'
argumenty pozycyjne: ([2, 3, 4],)
argumenty nazwane: {'wagi': [1, 2, 4], 'dokladnosc': 3}
wynik = 3.429
```

Out[62]:
```
3.429
```

Funkcja opakowująca przyjmie poniższe argumenty, ale ich zgodność z parametrami sprawdzi dopiero wywoływana funkcja `srednia`. Ostatnia liczba ma oznaczać dokładność, a została podana pozycyjnie. Funkcja `srednia` wymaga przekazania jej jako argumentu nazwanego `dokladnosc`, więc zgłosi wyjątek `TypeError`:

In [63]:
```
wywolaj_funkcje(srednia, [2, 3, 4], [3, 4, 5], 2)
```

```
wywołanie funkcji 'srednia'
argumenty pozycyjne: ([2, 3, 4], [3, 4, 5], 2)
argumenty nazwane: {}
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[63], line 1
----> 1 wywolaj_funkcje(srednia, [2, 3, 4], [3, 4, 5], 2)

Cell In[58], line 5, in wywolaj_funkcje(f, *args, **kwargs)
      1 def wywolaj_funkcje(f, *args, **kwargs):
      2     print(f"wywołanie funkcji '{f.__name__}'")
      3     print(f"argumenty pozycyjne: {args}")
      4     print(f"argumenty nazwane: {kwargs}")
----> 5     wynik = f(*args, **kwargs)
      6     print(f"wynik = {wynik}")
      7     return wynik

TypeError: srednia() takes from 1 to 2 positional arguments but 3 were given
```

Pokrewne zastosowania gwiazdek

Poniższe mechanizmy są blisko związane z poznanymi zastosowaniami gwiazdek, ale nie dotyczą już parametrów ani argumentów funkcji. Używamy ich przy przypisaniu oraz przy tworzeniu list i słowników.

Gwiazdka po lewej stronie przypisania

Podobnie jak parametr z gwiazdką pozwala otrzymać krotkę argumentów, tak gwiazdka przy przypisaniu pozwala otrzymać listę części elementów sekwencji:

In [64]:
```
pierwszy, *srodek, ostatni = [1, 2, 3, 4, 5]
```

In [65]:
```
pierwszy
```

Out[65]:
```
1
```

In [66]:
```
srodek
```

Out[66]:
```
[2, 3, 4]
```

In [67]:
```
ostatni
```

Out[67]:
```
5
```

Rozpakowanie w listach i słownikach

Podobny zapis pozwala wstawić elementy sekwencji do nowej listy i wpisy słownika do nowego słownika. Nie jest ograniczony do wywołań funkcji.

In [68]:
```
a = [1, 2, 3]
b = [0, 0, *a, 0, 0, *a, *range(10)]
```

In [69]:
```
b
```

Out[69]:
```
[0, 0, 1, 2, 3, 0, 0, 1, 2, 3, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

In [70]:
```
d = {"a": 1, "b": 2, "c": 3}
e = {"d": 4, "e": 5}
```

In [71]:
```
{**d, **e, "f": 6}
```

Out[71]:
```
{'a': 1, 'b': 2, 'c': 3, 'd': 4, 'e': 5, 'f': 6}
```

In [72]:
```
d
```

Out[72]:
```
{'a': 1, 'b': 2, 'c': 3}
```

In [73]:
```
e
```

Out[73]:
```
{'d': 4, 'e': 5}
```

W literałach słowników późniejszy wpis o tym samym kluczu zastępuje wcześniejszy. W wywołaniu funkcji dwukrotne podanie wartości tego samego parametru jest natomiast błędem.

In [74]:
```
{**{"a": 1}, **{"a": 2}}
```

Out[74]:
```
{'a': 2}
```